



Date: 13 / 02 / 2026

Lab Practical #9:

Implementation of Block Cipher Operation.

1. Electronic Code Book - ECB
2. Cipher Block Chaining Mode - CBC
3. Cipher Feedback Mode - CFB
4. Output Feedback Mode - OFB
5. Counter Mode - CTR

Program:

```
// 1. Electronic Code Book - ECB
// 1. Electronic Code Book - ECB

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

void xorBlock(char *input, char *key, char *output)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        output[i] = input[i] ^ key[i];
}

void ECB(char *input, char *key, char *output, int length)
{
    for(int i = 0; i < length; i += BLOCK_SIZE)
        xorBlock(input + i, key, output + i);
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";

    printf("Enter plaintext (multiple of 8): ");
    scanf("%s", plaintext);

    int length = strlen(plaintext);

    ECB(plaintext, key, ciphertext, length);
    ciphertext[length] = '\0'; // Fix

    ECB(ciphertext, key, decrypted, length);
    decrypted[length] = '\0'; // Fix
```



Date: 13 / 02 / 2026

```
printf("\nEncrypted: %s", ciphertext);  
printf("\nDecrypted: %s", decrypted);  
  
return 0;  
}
```

Output:

```
Enter plaintext (multiple of 8): ABCDEFGH  
Encrypted: pppppppp  
Decrypted: ABCDEFGH
```

Date: 13 / 02 / 2026

Program:

```
// 2. Cipher Block Chaining Mode - CBC

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

// Common XOR function
void xorBlock(char *a, char *b, char *result)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        result[i] = a[i] ^ b[i];
}

// CBC Encryption
void CBC_Encrypt(char *input, char *key, char *iv, char *output, int length)
{
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        xorBlock(input + i, iv, temp);
        xorBlock(temp, key, output + i);
        memcpy(iv, output + i, BLOCK_SIZE);
    }
}

// CBC Decryption
void CBC_Decrypt(char *input, char *key, char *iv, char *output, int length)
{
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        xorBlock(input + i, key, temp);
        xorBlock(temp, iv, output + i);
        memcpy(iv, input + i, BLOCK_SIZE);
    }
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";
    char iv[BLOCK_SIZE] = "87654321";
```

Date: 13 / 02 / 2026

```
char original_iv[BLOCK_SIZE];

printf("Enter plaintext (multiple of 8): ");
scanf("%s", plaintext);

int length = strlen(plaintext);

// Save original IV
memcpy(original_iv, iv, BLOCK_SIZE);

// Encrypt
CBC_Encrypt(plaintext, key, iv, ciphertext, length);
ciphertext[length] = '\0'; // Null terminate

// Reset IV before decryption
memcpy(iv, original_iv, BLOCK_SIZE);

// Decrypt
CBC_Decrypt(ciphertext, key, iv, decrypted, length);
decrypted[length] = '\0'; // Null terminate

printf("\nEncrypted: %s", ciphertext);
printf("\nDecrypted: %s", decrypted);

return 0;
}
```

Output:

```
Enter plaintext (multiple of 8): ABCDEFGH

Encrypted: HGFEDCBA
Decrypted: ABCDEFGH
```

Program:

```
// 3. Cipher Feedback Mode - CFB

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

// Common XOR function
void xorBlock(char *a, char *b, char *result)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        result[i] = a[i] ^ b[i];
}

// CFB Mode (Same for Encrypt & Decrypt)
void CFB(char *input, char *key, char *iv, char *output, int length)
{
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        xorBlock(iv, key, temp);           // Encrypt IV
        xorBlock(input + i, temp, output + i); // XOR with plaintext/ciphertext
        memcpy(iv, output + i, BLOCK_SIZE); // Update IV
    }
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";
    char iv[BLOCK_SIZE] = "87654321";
    char original_iv[BLOCK_SIZE];

    printf("Enter plaintext (multiple of 8): ");
    scanf("%s", plaintext);

    int length = strlen(plaintext);

    // Save original IV
    memcpy(original_iv, iv, BLOCK_SIZE);

    // Encrypt
```

Date: 13 / 02 / 2026

```
CFB(plaintext, key, iv, ciphertext, length);
ciphertext[length] = '\0';

// Reset IV before decryption
memcpy(iv, original_iv, BLOCK_SIZE);

// Decrypt (same function)
CFB(ciphertext, key, iv, decrypted, length);
decrypted[length] = '\0';

printf("\nEncrypted: %s", ciphertext);
printf("\nDecrypted: %s", decrypted);

return 0;
}
```

Output:

```
Enter plaintext (multiple of 8): ABCDEDGH

Encrypted: HGFEDABA
Decrypted: ABCDEDGH
```

Program:

```
// 4. Output Feedback Mode - OFB

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

// Common XOR function
void xorBlock(char *a, char *b, char *result)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        result[i] = a[i] ^ b[i];
}

// OFB Mode (Same for Encrypt & Decrypt)
void OFB(char *input, char *key, char *iv, char *output, int length)
{
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        xorBlock(iv, key, temp);
        memcpy(iv, temp, BLOCK_SIZE);
        xorBlock(input + i, iv, output + i);
    }
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";
    char iv[BLOCK_SIZE] = "87654321";
    char original_iv[BLOCK_SIZE];

    printf("Enter plaintext (multiple of 8): ");
    scanf("%s", plaintext);

    int length = strlen(plaintext);

    memcpy(original_iv, iv, BLOCK_SIZE);

    // Encrypt
    OFB(plaintext, key, iv, ciphertext, length);
```



Date: 13 / 02 / 2026

```
ciphertext[length] = '\\0';

// Reset IV
memcpy(iv, original_iv, BLOCK_SIZE);

// Decrypt (same function)
OFB(ciphertext, key, iv, decrypted, length);
decrypted[length] = '\\0';

printf("\\nEncrypted: %s", ciphertext);
printf("\\nDecrypted: %s", decrypted);

return 0;
}
```

Output:

```
Enter plaintext (multiple of 8): SDERFTCY

Encrypted: ZA@SGQFP
Decrypted: SDERFTCY
```

Program:

```
// 5. Counter Mode - CTR

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

void xorBlock(char *a, char *b, char *result)
{
    for(int i = 0; i < BLOCK_SIZE; i++)
        result[i] = a[i] ^ b[i];
}

void CTR(char *input, char *key, char *nonce, char *output, int length)
{
    char counterBlock[BLOCK_SIZE];
    char temp[BLOCK_SIZE];

    for(int i = 0; i < length; i += BLOCK_SIZE)
    {
        memcpy(counterBlock, nonce, BLOCK_SIZE);
        counterBlock[BLOCK_SIZE - 1] += i / BLOCK_SIZE;

        xorBlock(counterBlock, key, temp);
        xorBlock(input + i, temp, output + i);
    }
}

int main()
{
    char plaintext[64], ciphertext[64], decrypted[64];
    char key[BLOCK_SIZE] = "12345678";
    char nonce[BLOCK_SIZE] = "00000000";

    printf("Enter plaintext (multiple of 8): ");
    scanf("%s", plaintext);

    int length = strlen(plaintext);

    CTR(plaintext, key, nonce, ciphertext, length);
    CTR(ciphertext, key, nonce, decrypted, length);

    printf("\nEncrypted: %s", ciphertext);
}
```



Date: 13 / 02 / 2026

```
printf("\nDecrypted: %s", decrypted);  
  
return 0;  
}
```

Output:

```
Enter plaintext (multiple of 8): VXYUBI  
Encrypted: WZZQGO  
Decrypted: VXYUBI
```